

Web Privacy — TP1: Stateful Tracking

Lab Report

Zineb EL ANSARI

September 23, 2026

Contents

1	Setting Up the Environment	2
2	Running the First Website	3
3	Inspecting HTTP Headers	4
4	Manipulating Cookies	4
5	Challenge 1 — Third-Party Tracking	6
6	Challenge 2 — First-Party Analytics	8

Introduction

This report documents the TP on *stateful tracking*. The goal was to build a small web prototype able to recognize a visitor's browser (Alice) across multiple HTTP requests using cookies, and then to observe how this mechanism is exploited by third-party trackers and analytics services.

Each section below follows the same order as the TP handout, with the terminal/browser evidence collected during the session and the answers to the questions asked.

1 Setting Up the Environment

The Python virtual environment was created and activated, then Flask was installed inside it.

```
(base) C:\Users\WIN 11 25H2>cd "C:\Users\WIN 11 25H2\Desktop\statefulTracking"

(base) C:\Users\WIN 11 25H2\Desktop\statefulTracking>python --version
Python 3.14.6

(base) C:\Users\WIN 11 25H2\Desktop\statefulTracking>python -m venv venv

(base) C:\Users\WIN 11 25H2\Desktop\statefulTracking>venv\Scripts\activate

(venv) (base) C:\Users\WIN 11 25H2\Desktop\statefulTracking>pip install flask
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.5.0-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from flask)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from flask)
  Downloading markupsafe-3.0.3-cp314-cp314-win_amd64.whl.metadata (2.8 kB)
Collecting werkzeug>=3.1.0 (from flask)
  Downloading werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Downloading flask-3.1.3-py3-none-any.whl (103 kB)
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.5.0-py3-none-any.whl (125 kB)
```

Figure 1: Creating and activating the virtual environment, then installing Flask.

```
Downloading click-8.5.0-py3-none-any.whl (125 kB)
Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
Downloading markupsafe-3.0.3-cp314-cp314-win_amd64.whl (15 kB)
Downloading werkzeug-3.1.8-py3-none-any.whl (226 kB)
Installing collected packages: markupsafe, itsdangerous, click, blinker, werkzeug, jinja2, flask
Successfully installed blinker-1.9.0 click-8.5.0 flask-3.1.3 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 werkzeug-3.1.8

[notice] A new release of pip is available: 26.1.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip

(venv) (base) C:\Users\WIN 11 25H2\Desktop\statefulTracking>flask--version
'flask--version' n'est pas reconnu en tant que commande interne
ou externe, un programme exécutable ou un fichier de commandes.

(venv) (base) C:\Users\WIN 11 25H2\Desktop\statefulTracking>flask --version
'C:\Users\WIN 11 25H2\Desktop\statefulTracking\venv\Scripts\flask.exe' a été bloqué par la stratégie Device Guard de votre organisation.
Pour plus d'informations, contactez la personne chargée du support.

(venv) (base) C:\Users\WIN 11 25H2\Desktop\statefulTracking>python -c "import flask; print(flask.__version__)"
<string>:1: DeprecationWarning: The '__version__' attribute is deprecated and will be removed in Flask 3.2. Use feature detection or 'importlib.metadata.version("flask")' instead.
3.1.3
```

Figure 2: End of the Flask installation and version check.

2 Running the First Website

The project structure (`app.py` and `templates/index.html`) was created, the Flask server was started on port 8000, and the site was opened in the browser at `http://lab.test:8000` (after adding the corresponding entry to the `hosts` file).

```
(venv) (base) C:\Users\WIN 11 25H2\Desktop\statefulTracking>type nul > app.py

(venv) (base) C:\Users\WIN 11 25H2\Desktop\statefulTracking>mkdir templates

(venv) (base) C:\Users\WIN 11 25H2\Desktop\statefulTracking>type nul > templates\index.html

(venv) (base) C:\Users\WIN 11 25H2\Desktop\statefulTracking>tree /F
Structure du dossier
Le numéro de série du volume est 000000B5 D678:1B4B
C:.
├── app.py
└── templates
    └── index.html

venv
```

Figure 3: Folder structure of `statefulTracking` (`app.py`, `templates/index.html`).

```
(venv) (base) C:\Users\WIN 11 25H2\Desktop\statefulTracking>notepad templates\index.html

(venv) (base) C:\Users\WIN 11 25H2\Desktop\statefulTracking>notepad app.py

(venv) (base) C:\Users\WIN 11 25H2\Desktop\statefulTracking>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:8000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 133-271-995
127.0.0.1 - - [22/Sep/2026 11:22:33] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2026 11:22:33] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [22/Sep/2026 11:22:42] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2026 11:22:46] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2026 11:27:12] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [22/Sep/2026 11:27:13] "GET /favicon.ico HTTP/1.1" 404 -
```

Figure 4: Editing the files, then starting the Flask server; the server log shows the incoming requests.

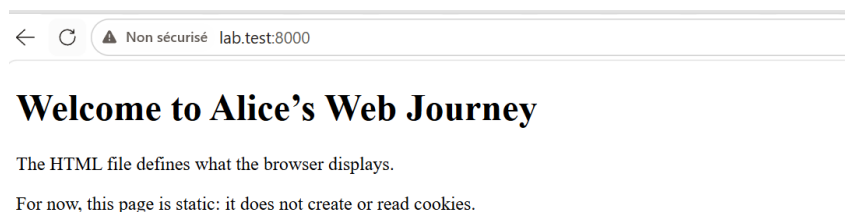


Figure 5: *Alice's Web Journey* page displayed in the browser at `lab.test:8000`.

3 Inspecting HTTP Headers

A custom header (`X-Lab-Message`) was added to the server response, and the request/response headers were displayed both in the browser DevTools and in the Flask terminal.

▼ General	
Request URL	http://lab.test:8000/
Request Method	GET
Status Code	● 200 OK
Remote Address	127.0.0.1:8000
Referrer Policy	strict-origin-when-cross-origin
▼ Response headers ☐ Raw	
Connection	close
Content-Length	282
Content-Type	text/html; charset=utf-8
Date	Tue, 22 Sep 2026 11:42:31 GMT
Server	Werkzeug/3.1.8 Python/3.14.6
X-Lab-Message	Hello from the Flask server

Figure 6: Browser Network tab: HTTP response headers, including `X-Lab-Message`.

```
--- HTTP REQUEST HEADERS---
Host: lab.test:8000
Connection: keep-alive
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36 Edg/153.0.0.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate
Accept-Language: fr,fr-FR;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6

--- HTTP RESPONSE HEADERS---
Content-Type: text/html; charset=utf-8
Content-Length: 282
X-Lab-Message: Hello from the Flask server
```

Figure 7: Request and response headers printed in the Flask server terminal.

4 Manipulating Cookies

After adding the creation of an `aid` cookie (a unique identifier generated with `secrets.token_hex(8)`) with no special attribute, the handout asked:

Is the cookie still present? Does it have the same identifier?

No. Since no `Max-Age/Expires` attribute is set, the `aid` cookie created by `response.set_cookie()` is a *session cookie*: it only lasts as long as the browser stays open. After closing and reopening the browser, the cookie is gone — the server no longer receives an `aid` cookie, treats the visitor as new, and generates a **new, different identifier**.

The following four cookie attributes were then tested one by one: `Max-Age`, a plain/default cookie (session), `HttpOnly`, and `SameSite=Strict`.

The screenshot shows the Chrome DevTools Application tab with the 'Application' pane open. The left sidebar shows the 'Storage' section expanded, with 'Cookies' selected. The main pane displays a table of cookies. The 'aid' cookie is selected, showing its details in the table below.

Name	Value	Domain	Path	Expires	Size	HttpO...	Secure	SameS...	Partiti...	Cross ...	Priority
aid	97e6d5dda75ba033	lab.test	/	2026-0...	19						Medium

Figure 8: `aid` cookie with the `Max-Age` attribute set (an explicit expiration date now appears in the *Expires* column).

The screenshot shows the Chrome DevTools Application tab with the 'Application' pane open. The left sidebar shows the 'Storage' section expanded, with 'Cookies' selected. The main pane displays a table of cookies. The 'aid' cookie is selected, showing its details in the table below.

Name	Value	Domain	Path	Expires	Size	HttpO...	Secure	SameS...	Partiti...	Cross ...	Priority
aid	f189c7c2053b5a35	lab.test	/	Session	19						Medi...

Figure 9: Plain / default `aid` cookie, no special attribute: *Expires* = *Session*.

The screenshot shows the Chrome DevTools Application tab with the 'Application' pane open. The left sidebar shows the 'Storage' section expanded, with 'Cookies' selected. The main pane displays a table of cookies. The 'aid' cookie is selected, showing its details in the table below.

Name	Value	Domain	Path	Expires	Size	HttpO...	Secure	SameS...	Partiti...	Cross ...	Priority
aid	a3a95906c845e127	lab.test	/	Session	19	✓					Medium

Figure 10: `aid` cookie with `HttpOnly` checked.

The screenshot shows the Chrome DevTools Application tab with the 'Application' pane open. The left sidebar shows the 'Storage' section expanded, with 'Cookies' selected. The main pane displays a table of cookies. The 'aid' cookie is selected, showing its details in the table below.

Name	Value	Domain	Path	Expires	Size	HttpO...	Secure	SameS...	Partiti...	Cross ...	Priority
aid	d60135202d38e8a4	lab.test	/	Session	19			Strict			Medium

Figure 11: `aid` cookie with `SameSite` = `Strict`.

Observations and explanation for each attribute:

- **Max-Age**: setting `max_age` (or `expires`) in `set_cookie()` gives the cookie a concrete expiration date (the *Expires* column now shows a date instead of *Session*). It becomes a *persistent cookie*: it survives closing the browser, unlike the default session cookie.
- **Plain / default cookie**: with no attribute, the cookie has no expiration date (*Session*) and disappears when the browser closes, exactly as observed in the question above.
- **HttpOnly**: the *HttpOnly* box is checked in DevTools. The cookie is still sent normally with every HTTP request, but it can no longer be read via `document.cookie` in JavaScript — this protects it against theft by a malicious script (XSS).
- **SameSite = Strict**: the *SameSite* column now shows *Strict*. The cookie is only sent when the request originates from the same site; it is no longer attached to requests triggered from a third-party site, which limits cross-site tracking and CSRF attacks.

5 Challenge 1 — Third-Party Tracking

Two independent publisher sites were created (`publisher-one.lab.test`, technology content; `publisher-two.lab.test`, travel content), along with one third-party tracker (`tracker-one.lab.test`) embedded in an `iframe` on both publishers.

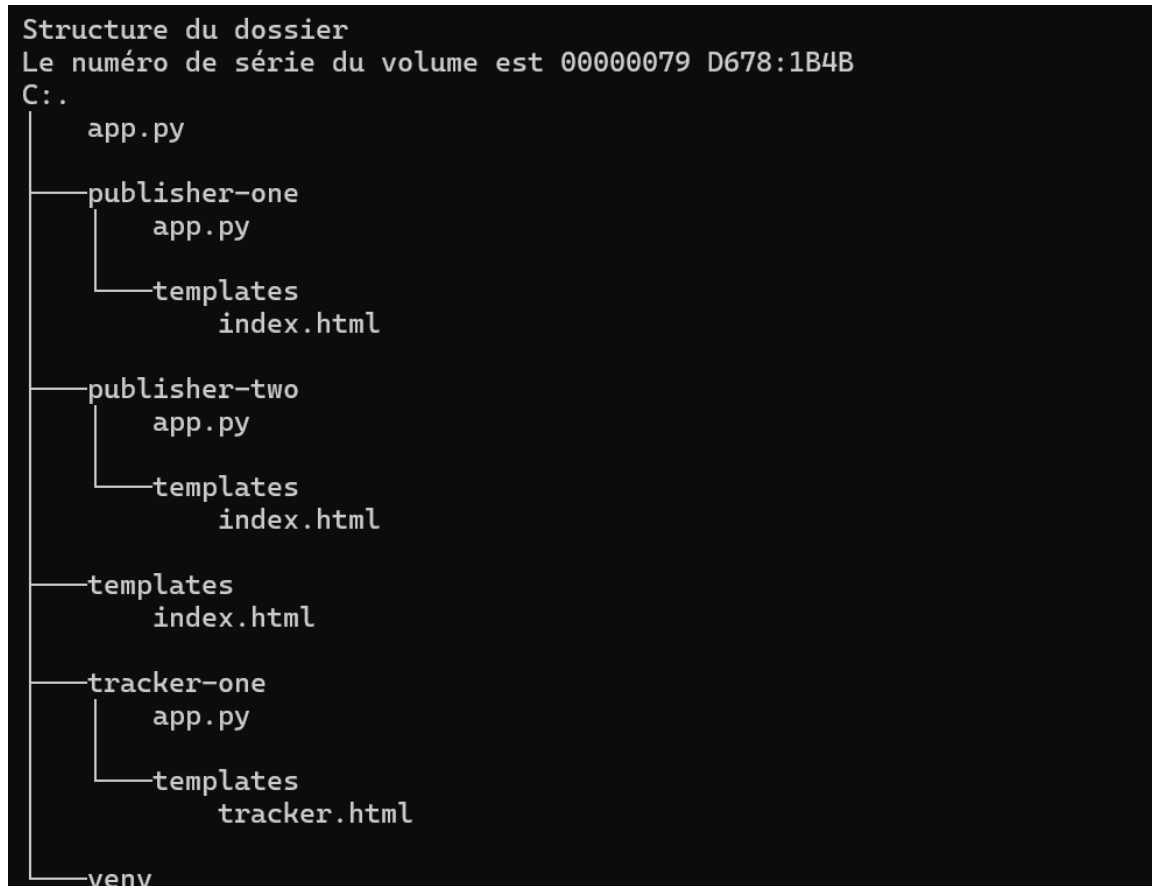


Figure 12: Project tree: `publisher-one`, `publisher-two` and `tracker-one`, each with its own `app.py` and `templates`.

```

(envv) (base) C:\Users\WIN 11 25H2\Desktop\statefulTracking>mkdir publisher-one
(envv) (base) C:\Users\WIN 11 25H2\Desktop\statefulTracking>mkdir publisher-one\templates
(envv) (base) C:\Users\WIN 11 25H2\Desktop\statefulTracking>mkdir publisher-two
(envv) (base) C:\Users\WIN 11 25H2\Desktop\statefulTracking>mkdir publisher-two\templates
(envv) (base) C:\Users\WIN 11 25H2\Desktop\statefulTracking>mkdir tracker-one
(envv) (base) C:\Users\WIN 11 25H2\Desktop\statefulTracking>mkdir tracker-one\templates
(envv) (base) C:\Users\WIN 11 25H2\Desktop\statefulTracking>type nul > publisher-one\app.py
(envv) (base) C:\Users\WIN 11 25H2\Desktop\statefulTracking>type nul > publisher-one\templates\index.html
(envv) (base) C:\Users\WIN 11 25H2\Desktop\statefulTracking>type nul > publisher-two\app.py
(envv) (base) C:\Users\WIN 11 25H2\Desktop\statefulTracking>type nul > publisher-two\templates\index.html
(envv) (base) C:\Users\WIN 11 25H2\Desktop\statefulTracking>type nul > tracker-one\app.py
(envv) (base) C:\Users\WIN 11 25H2\Desktop\statefulTracking>type nul > tracker-one\templates\tracker.html
(envv) (base) C:\Users\WIN 11 25H2\Desktop\statefulTracking>

```

Figure 13: Creating the folders and files for the two publishers and the tracker.

The screenshot shows the Chrome DevTools Application tab for the URL `publisher-one.lab.test:8001`. The left sidebar shows the 'Storage' section with 'Cookies' expanded. Two cookies are listed: one for `http://publisher-one.lab.test` and another for `http://tracker-one.lab.test`. The cookie for `http://tracker-one.lab.test` is selected, showing its details in the main pane.

Name	Value	Domain	Path	Expire...	Size	HttpO...	Secure	Same...	Partiti...	Cross ...	Priority
aid	a7917b80e36e1fa04cc0baad7dc97b62	tracke...	/	2027-...	35						Medi...

Figure 14: `publisher-one.lab.test:8001` (Technology World): the `aid` cookie belongs to domain `tracker-one.lab.test` (third-party cookie), value `a7917b80e36e1fa04cc0baad7dc97b62`.

The screenshot shows the Chrome DevTools Application tab for the URL `publisher-two.lab.test:8002`. The left sidebar shows the 'Storage' section with 'Cookies' expanded. Two cookies are listed: one for `http://publisher-two.lab.test` and another for `http://tracker-one.lab.test`. The cookie for `http://tracker-one.lab.test` is selected, showing its details in the main pane.

Name	Value	Domain	Path	Expire...	Size	HttpO...	Secure	Same...	Partiti...	Cross ...	Priority
aid	a7917b80e36e1fa04cc0baad7dc97b62	tracke...	/	2027-...	35						Medi...

Figure 15: `publisher-two.lab.test:8002` (Travel World): the same third-party `aid` cookie from `tracker-one.lab.test`, with the exact same value `a7917b80e36e1fa04cc0baad7dc97b62`.

Reconstructed profile (chronological order):

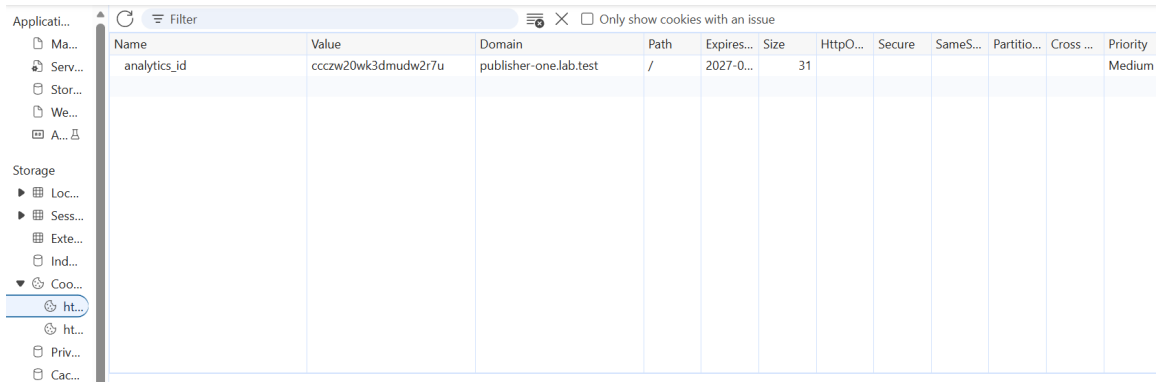
The `aid` cookie set by the tracker's `iframe` belongs to `tracker-one.lab.test`, not to the publisher shown in the address bar: it is a **third-party cookie**. It carries the **exact same value** on both `publisher-one.lab.test` and `publisher-two.lab.test` — the browser sends the same identifier to the tracker no matter which top-level site is being visited.

Identifier (aid)	Publisher visited	Page
a7917b80e36e1fa04cc0baad7dc97b62	publisher-one.lab.test	/ (Technology World)
a7917b80e36e1fa04cc0baad7dc97b62	publisher-two.lab.test	/ (Travel World)

Because the same persistent identifier is shared across both sites, **tracker-one** can build a **single cross-site profile** for this browser: it knows that the same visitor (identified by `aid`) looked at technology articles and then at travel articles — even though the two publishers are completely independent and never exchange any information directly. This is exactly the principle of **third-party tracking**.

6 Challenge 2 — First-Party Analytics

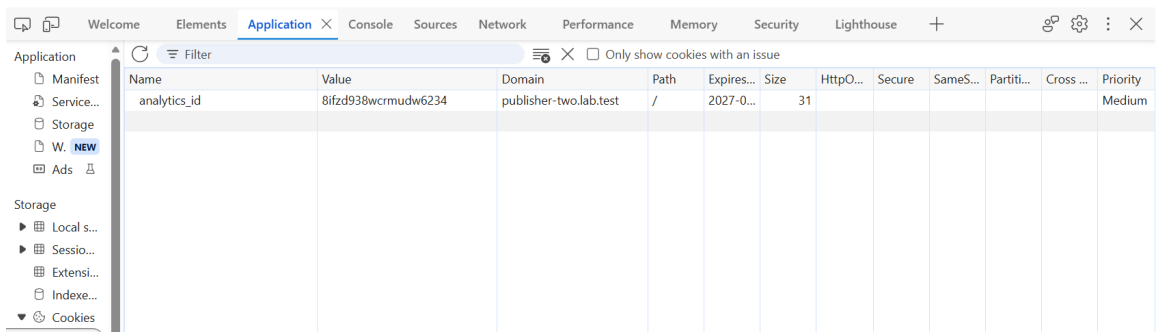
A script `analytics.js`, served from a separate analytics domain, was included on both publishers. This script itself creates the cookie via `document.cookie` (first-party context), then sends the identifier to the analytics server through a `/collect` request.



The screenshot shows the Chrome DevTools Application tab with the 'Cookies' section expanded. A table lists the cookies for the domain publisher-one.lab.test. The 'analytics_id' cookie is highlighted, showing its value as 'ccczw20wk3dmudw2r7u'.

Name	Value	Domain	Path	Expires...	Size	HttpO...	Secure	SameS...	Partiti...	Cross ...	Priority
analytics_id	ccczw20wk3dmudw2r7u	publisher-one.lab.test	/	2027-0...	31						Medium

Figure 16: `analytics_id` cookie seen on `publisher-one.lab.test`: domain `publisher-one.lab.test`, value `ccczw20wk3dmudw2r7u`.



The screenshot shows the Chrome DevTools Application tab with the 'Cookies' section expanded. A table lists the cookies for the domain publisher-two.lab.test. The 'analytics_id' cookie is highlighted, showing its value as '8ifzd938wcrmdw6234'.

Name	Value	Domain	Path	Expires...	Size	HttpO...	Secure	SameS...	Partiti...	Cross ...	Priority
analytics_id	8ifzd938wcrmdw6234	publisher-two.lab.test	/	2027-0...	31						Medium

Figure 17: `analytics_id` cookie seen on `publisher-two.lab.test`: domain `publisher-two.lab.test`, value `8ifzd938wcrmdw6234` (different from the one above).

```

--- ANALYTICS REQUEST ---
ID: ccczw20wk3dmudw2r7u
Publisher: publisher-one.lab.test
Page: /
127.0.0.1 - - [23/Sep/2026 09:22:50] "GET /collect?id=ccczw20wk3dmudw2r7u&publisher=publisher-one.lab.test&page=/ HTTP/1.1" 200 -
127.0.0.1 - - [23/Sep/2026 09:23:34] "GET /static/analytics.js HTTP/1.1" 200 -

--- ANALYTICS REQUEST ---
ID: ccczw20wk3dmudw2r7u
Publisher: publisher-one.lab.test
Page: /
127.0.0.1 - - [23/Sep/2026 09:23:34] "GET /collect?id=ccczw20wk3dmudw2r7u&publisher=publisher-one.lab.test&page=/ HTTP/1.1" 200 -
127.0.0.1 - - [23/Sep/2026 09:23:58] "GET /static/analytics.js HTTP/1.1" 200 -

--- ANALYTICS REQUEST ---
ID: 8ifzd938wcrmdw6234
Publisher: publisher-two.lab.test
Page: /
127.0.0.1 - - [23/Sep/2026 09:23:58] "GET /collect?id=8ifzd938wcrmdw6234&publisher=publisher-two.lab.test&page=/ HTTP/1.1" 200 -

```

Figure 18: Analytics server log: incoming `/collect` requests, with the identifier and the originating publisher.

Name	Status	Type	Initiator	Size	Time	Fulfilled by
publisher-one.lab.test	200	document	Other	758 B	7 ms	
analytics.js	200	script	(index):22	1.5 kB	47 ms	
tracker?publisher=publisher-one&page=home		document	(index):20	311 kB	31 ms	
collect?id=ccczw20wk3dmudw2r7u&publisher=publisher-one.l...	200	fetch	analytics.js:44	0 B	18 ms	

Figure 19: Browser Network tab on a publisher page: loading of the document, `analytics.js`, the tracker iframe, and the `collect?id=...&publisher=...` request.

Are the two publishers assigned the same identifier or different identifiers?

Different identifiers: `ccczw20wk3dmudw2r7u` for `publisher-one.lab.test` and `8ifzd938wcrmdw6234` for `publisher-two.lab.test` (see the figures above).

Why does this behaviour occur?

`analytics.js` runs in the *first-party* context of the page that includes it, so the `analytics_id` cookie it creates with `document.cookie` is attached to the current publisher's domain (`publisher-one.lab.test` or `publisher-two.lab.test`), not to the analytics service's domain. Cookies are isolated per domain, so the cookie created on `publisher-one.lab.test` is neither readable nor sent on `publisher-two.lab.test`. Each site therefore generates its own identifier, independently of the other.

Does the analytics server automatically receive the first-party cookies created on the publishers?

No. These cookies belong to each publisher’s own domain, not to `analytics.test`, so the browser never automatically attaches them to requests sent to the analytics server. This is why the script must explicitly read the identifier from `document.cookie` and send it “by hand” as a URL parameter (`/collect?id=...&publisher=...&page=...`), as shown in the server log above.

Does the analytics service build one combined profile or several separate profiles? Explain the result.

The server log (figure above) shows two distinct identifiers: `ccczw20wk3dmudw2r7u` tied to `publisher-one.lab.test`, and `8ifzd938wcrmudw6234` tied to `publisher-two.lab.test`. The analytics service therefore builds **two separate profiles**, one per publisher, rather than a single cross-site profile.

Identifier	Associated publisher
<code>ccczw20wk3dmudw2r7u</code>	<code>publisher-one.lab.test</code>
<code>8ifzd938wcrmudw6234</code>	<code>publisher-two.lab.test</code>

This is the direct opposite of Challenge 1: because the `analytics_id` cookie is *first-party* (created separately by each site), the analytics service cannot link the two identifiers together and only sees isolated visits per site. A *third-party* tracker, as in Challenge 1, instead shares the same cookie (and thus the same identifier) on every site that embeds it, which is exactly what lets it build one unified cross-site profile.

Conclusion

This lab put into practice the core mechanisms of *stateful tracking* based on HTTP cookies: persistence control (`Max-Age`), protection against JavaScript theft (`HttpOnly`), limiting cross-site transmission (`SameSite`), and the fundamental difference between a **third-party** cookie (which lets a tracker follow the same user across several independent sites) and a **first-party** cookie (isolated per domain, which only allows a per-site profile to be built).